

IF2224 - Teori Bahasa Formal dan Otomata

Tugas Besar

Milestone 2 : Syntax Analysis

16 April 2026 - 7 Mei 2026



Mirza Tsabita Wafa'ana 13524114

Nathanael Shane Bennet 13524119

Faris Wirakusuma Triawan 13524130

Jonathan Harijadi 13524144

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

BAB 1 : Teori Singkat.....	3
1.1. Kompilator (Compiler).....	3
1.2. Syntax Analysis.....	3
1.3. Recursive Descent.....	3
1.4. Parse Tree.....	3
BAB 2 : Perancangan & Implementasi.....	4
2.2 Implementasi Program.....	4
1. File node.hpp.....	4
2. File parsingTree.cpp dan parsingTree.hpp.....	5
BAB 3 : Pengujian.....	8
BAB 4 : Kesimpulan dan Saran.....	20
4.1 Kesimpulan.....	20
4.2 Saran.....	20
Lampiran.....	22
Referensi.....	23

BAB 1 : Teori Singkat

1.1. Kompilator (*Compiler*)

Kompilator merupakan suatu program yang bertugas menerjemahkan kode sumber (*source code*) yang ditulis dalam bahasa pemrograman tingkat tinggi menjadi kode yang dapat dieksekusi oleh mesin. Proses kompilasi secara umum terbagi menjadi beberapa tahap, yaitu analisis leksikal (*lexical analysis*), analisis sintaksis (*syntax analysis*), analisis semantik (*semantic analysis*), optimasi kode, dan pembangkitan kode target. Tahap pertama dan paling mendasar dari proses kompilasi adalah analisis leksikal, yang menjadi fokus utama pada proyek ini (Aho, Lam, Sethi, & Ullman, 2006).

1.2. Syntax Analysis

Analisis sintaksis (*parsing*) adalah fase kedua dalam hierarki kompilasi yang bertugas memvalidasi urutan token linier dari *lexical analyzer* untuk memastikan kesesuaiannya secara ketat dengan *Context-Free Grammar* (CFG) dari suatu bahasa pemrograman. Evaluasi struktural ini secara deterministik mengonversi himpunan token menjadi representasi memori hierarkis berupa *Parse Tree* atau *Abstract Syntax Tree* (AST) dengan memanfaatkan algoritma penelusuran tata bahasa, baik menggunakan pendekatan *top-down* maupun *bottom-up*; kegagalan pencocokan pada fase ini secara langsung menghasilkan *Syntax Error*.

1.3. Recursive Descent

Recursive Descent merupakan salah satu metode analisis sintaksis dengan pendekatan *top-down* yang diimplementasikan melalui sekumpulan prosedur rekursif untuk memproses simbol-simbol dalam tata bahasa (*grammar*). Dalam teknik ini, setiap non-terminal pada tata bahasa direpresentasikan menjadi sebuah fungsi atau prosedur dalam kode program. Parser akan mulai bekerja dari simbol start dan secara rekursif memanggil prosedur-prosedur lain yang sesuai dengan aturan produksi hingga seluruh token input berhasil dicocokkan atau ditemukan kesalahan sintaksis.

1.4. Parse Tree

Parse Tree adalah representasi grafis hierarkis yang mengilustrasikan bagaimana rangkaian token input diturunkan dari simbol start berdasarkan aturan produksi dalam tata bahasa. Dalam struktur ini, node internal merepresentasikan simbol non-terminal, sedangkan node daun (*leaf nodes*) merepresentasikan simbol terminal atau token. Keberhasilan parser dalam membangun Parse Tree yang lengkap dan valid menandakan bahwa kode sumber secara sintaksis benar. Hasil dari konstruksi pohon ini selanjutnya akan menjadi input krusial bagi tahap analisis semantik untuk memverifikasi tipe data dan logika keterkaitan antar simbol dalam program.

BAB 2 : Perancangan & Implementasi

2.2 Implementasi Program

Program dibuat menggunakan bahasa C++ dan implementasi syntax analyser berada di file-file berikut : parsingTree.cpp dan parsingTree.hpp, serta file node.hpp.

1. File node.hpp

File ini bertugas sebagai definisi struktur data fundamental yang akan menjadi *output* dari *node* dan *input* bagi *parser*. Didalam file ini terdapat enum yang berisikan token-token yang nantinya akan dibaca oleh syntax analyzer. Pemanggilan token dapat dilakukan melalui class node.

Berikut adalah enum dari nodenya:

```
enum NodeType {
    TOKEN_NODE,
    PROGRAM,
    PROGRAM_HEADER,
    DECLARATION_PART,
    CONST_DECLARATION,
    CONSTANT,
    TYPE_DECLARATION,
    VAR_DECLARATION,
    IDENTIFIER_LIST,
    TYPE,
    ARRAY_TYPE,
    RANGE,
    ENUMERATED,
    RECORD_TYPE,
    FIELD_LIST,
    FIELD_PART,
    SUBPROGRAM_DECLARATION,
    PROCEDURE_DECLARATION,
    FUNCTION_DECLARATION,
    BLOCK,
    FORMAL_PARAMETER_LIST,
    PARAMETER_GROUP,
    COMPOUND_STATEMENT,
    PARAMETER_GROUP,
    COMPOUND_STATEMENT,
    STATEMENT_LIST,
    STATEMENT,
    VARIABLE,
    COMPONENT_VARIABLE,
    INDEX_LIST,
    ASSIGNMENT_STATEMENT,
    IF_STATEMENT,
    CASE_STATEMENT,
    CASE_BLOCK,
    WHILE_STATEMENT,
    REPEAT_STATEMENT,
    FOR_STATEMENT,
    PROCEDURE_FUNCTION_CALL,
    PARAMETER_LIST,
    EXPRESSION,
    SIMPLE_EXPRESSION,
    TERM,
    FACTOR,
    RELATIONAL_OPERATOR,
    ADDITIVE_OPERATOR,
    MULTIPLICATIVE_OPERATOR
};
```

Berikut adalah implementasi class nodenya:

```
class Node {
private:
public:
    NodeType type;
    string lexeme;
    vector<std::unique_ptr<Node>> children;
    Node(NodeType t) : type(t), lexeme("") {}
    Node(NodeType t, std::string l) : type(t), lexeme(l) {}
    ~Node() = default;
    void addChild(unique_ptr<Node> child);
    void printDFS(std::ostream& os, std::string prefix = "", bool isLast = true, bool isRoot = true) const;
};

string nodeTypeToString(NodeType type);
```

2. File parsingTree.cpp dan parsingTree.hpp

Kedua file ini bertugas sebagai syntax analyzer utama dalam program ini. Menggunakan pola arsitektur deklarasi dan implementasi, kedua file ini mengimplementasikan algoritma Top-Down Recursive Descent Parsing secara deterministik untuk memvalidasi urutan pita token hasil lexical analyzer dan mengkonversikannya menjadi parse tree 2D.

State Management & Alokasi Memori:

```
class ParsingTree {
private:
    Lexer& lexer;
    Token currentToken;
    Token nextToken;
    std::unique_ptr<Node> root;
```

Bagian ini menyimpan state operasional kompilator selama fase sintaksis berjalan. Objek lexer diikat menggunakan referensi (&) untuk memastikan parser membaca dari mesin DFA yang sama tanpa menyalin data. Variabel currentToken dan nextToken bertindak sebagai buffer yang menyimpan token yang sedang dievaluasi. Kepemilikan struktur pohon berakar pada root yang menggunakan smart pointer (std::unique_ptr); arsitektur ini menjamin pembersihan memori (garbage collection) pada seluruh cabang di bawahnya berjalan otomatis saat kompilasi selesai atau terhenti akibat crash, mengeliminasi risiko memory leak.

Core Parsing Utilities (Utilitas Validasi Dasar):

```

void advance();
std::string tokenLabel(const Token& token) const;
std::unique_ptr<Node> accept(TokenType expectedType);
std::unique_ptr<Node> expect(TokenType expectedType);

```

Sekumpulan operasi primitif untuk memanipulasi pita token secara linier. Metode `advance()` memicu Lexer untuk menerima buffer dengan token valid berikutnya, secara otomatis mengabaikan spasi dan blok komentar. `accept()` melakukan validasi non-kritis; jika token cocok, pointer maju, jika tidak, operasi diabaikan tanpa interupsi. `expect()` adalah utilitas validasi absolut. Metode ini mewajibkan kecocokan token dengan spesifikasi tata bahasa. Kegagalan validasi pada titik ini merupakan pemicu mutlak (`throw runtime_error`) yang menghasilkan Syntax Error dan menghentikan kompilasi.

Predictive Lookahead (Himpunan FIRST)

```

bool isStatementStart(TokenType type) const;
bool isFactorStart(TokenType type) const;
bool isRelationalOperator(TokenType type) const;
bool isAdditiveOperator(TokenType type) const;
bool isMultiplicativeOperator(TokenType type) const;

```

Fungsi-fungsi prediktif ini merupakan representasi kode dari himpunan FIRST dalam teori Context-Free Grammar (CFG). Parser menggunakan fungsi boolean ini sebagai mekanisme lookahead evaluasi untuk menguji validitas jenis token sebelum mengeksekusi percabangan instruksi (`if-else` atau `while`). Implementasi ini memastikan parser tidak masuk ke dalam fungsi rekursif yang salah saat menemui ambiguitas operasional, menjaga sifat deterministik dari algoritma.

Recursive Descent Implementations (Aturan Produksi)

```

std::unique_ptr<Node> parseProgram();
std::unique_ptr<Node> parseDeclarationPart();
std::unique_ptr<Node> parseStatement();
std::unique_ptr<Node> parseExpression();
std::unique_ptr<Node> parseFactor();

```

Blok ini memuat sekumpulan metode rekursif internal yang dipetakan secara eksak (1:1) terhadap aturan tata bahasa spesifikasi. Ini adalah inti dari konstruksi hierarki; setiap

fungsi bertanggung jawab mengikat daun (token node) menjadi cabang memori, saling memanggil melalui Call Stack C++ dari tingkat makro struktur program (parseProgram), turun ke blok deklarasi dan statement, hingga level paling mikro berupa evaluasi presedensi operator matematika tunggal (parseFactor).

Public Application Programming Interface (API):

```
public:
    ParsingTree(Lexer& lexInstance);
    ~ParsingTree() = default;

    void build();
    void printToCLI();
    void exportToFile(const std::string& filename);
```

Antarmuka publik ini bertindak sebagai titik interaksi antara modul sintaksis dengan titik masuk utama (main.cpp). Konstruktor diinjeksikan secara Dependency Injection dengan instansi Lexer. Metode build() bertindak sebagai pemicu (trigger) utama untuk merangkai objek pohon mulai dari node puncak. Setelah kompilasi sintaksis berhasil, printToCLI() dan exportToFile() disediakan untuk melintasi graf memori secara Depth-First Search (DFS) guna merender struktur hierarki 2D secara visual ke layar terminal atau menyimpannya sebagai berkas rekam jejak (log).

BAB 3 : Pengujian

Input	Output
Case 1	
<pre>program Hello; var a, b, i: integer; function AddTen(x: integer): integer; begin AddTen := x + 10; end; begin a := 5; b := AddTen(a); writeln('Result = ', b); end.</pre>	<pre><program> ├── <program-header> │ ├── programsy │ ├── ident(Hello) │ └── semicolon ├── <declaration-part> │ ├── <var-declaration> │ │ ├── varsy │ │ ├── <identifier-list> │ │ │ ├── ident(a) │ │ │ ├── comma │ │ │ ├── ident(b) │ │ │ ├── comma │ │ │ └── ident(i) │ │ ├── colon │ │ ├── <type> │ │ │ └── ident(integer) │ │ └── semicolon │ └── <subprogram-declaration> │ ├── <function-declaration> │ │ ├── functionsy │ │ ├── ident(AddTen) │ │ ├── <formal-parameter-list> │ │ │ ├── lparent │ │ │ ├── <parameter-group> │ │ │ │ ├── <identifier-list> │ │ │ │ │ └── ident(x) │ │ │ │ ├── colon │ │ │ │ └── ident(integer) │ │ │ └── rparent │ │ ├── colon │ │ ├── ident(integer) │ │ └── semicolon │ └── <...></pre>


```

      ident(integer)
    | semicolon
    | <block>
    |   | <declaration-part>
    |   | <compound-statement>
    |   |   | beginsy
    |   |   | <statement-list>
    |   |   |   | <statement>
    |   |   |   |   | <assignment-statement>
    |   |   |   |   |   | <variable>
    |   |   |   |   |   |   | ident(AddTen)
    |   |   |   |   |   | becomes
    |   |   |   |   |   | <expression>
    |   |   |   |   |   |   | <simple-expression>
    |   |   |   |   |   |   |   | <term>
    |   |   |   |   |   |   |   |   | <factor>
    |   |   |   |   |   |   |   |   |   | ident(x)
    |   |   |   |   |   |   |   |   | <additive-operator>
    |   |   |   |   |   |   |   |   |   | plus
    |   |   |   |   |   |   |   |   | <term>
    |   |   |   |   |   |   |   |   |   | <factor>
    |   |   |   |   |   |   |   |   |   | intcon(10)
    |   |   |   |   |   |   | semicolon
    |   |   |   | endsy
    |   | semicolon
    |   endsy
    | semicolon
  | <compound-statement>
  |   | beginsy
  |   | <statement-list>
  |   |   | <statement>
  |   |   |   | <assignment-statement>
  |   |   |   |   | <variable>
  |   |   |   |   |   | ident(a)
  |   |   |   |   | becomes
  |   |   |   |   | <expression>

```

```

└─ becomes
  └─ <expression>
    └─ <simple-expression>
      └─ <term>
        └─ <factor>
          └─ intcon(5)

└─ semicolon
└─ <statement>
  └─ <assignment-statement>
    └─ <variable>
      └─ ident(b)
    └─ becomes
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ <procedure/function-call>
                └─ ident(AddTen)
                └─ lparent
                └─ <parameter-list>
                  └─ <expression>
                    └─ <simple-expression>
                      └─ <term>
                        └─ <factor>
                          └─ ident(a)
                └─ rparent

└─ semicolon
└─ <statement>
  └─ <procedure/function-call>
    └─ ident(writeln)
    └─ lparent
    └─ <parameter-list>
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ string('Result = ')
      └─ comma
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ ident(b)
    └─ rparent

└─ semicolon
└─ endsy
└─ period

```

Case 2

```

program LogicTest;
var
  x, y: integer;

  check: boolean;
begin
  x := 10;
  y := 20;

  if (x <> y) and (x <= y) then
  begin
    check := true;
  end
  else
  begin
    check := false;
  end;
end.

```

```

<program>
├─ <program-header>
│   ├── programsy
│   ├── ident(LogicTest)
│   └─ semicolon
├─ <declaration-part>
│   └─ <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident(x)
│       │   ├── comma
│       │   └─ ident(y)
│       ├── colon
│       ├── <type>
│       │   └─ ident(integer)
│       ├── semicolon
│       ├── <identifier-list>
│       │   └─ ident(check)
│       ├── colon
│       ├── <type>
│       │   └─ ident(boolean)
│       └─ semicolon
├─ <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   └─ <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └─ ident(x)
│   │   │       ├── becomes
│   │   │       └─ <expression>
│   │   │           └─ <simple-expression>
│   │   │               └─ <term>
│   │   │                   └─ <factor>
│   │   │                       └─ intcon(10)
│   │   ├── semicolon
│   │   ├── <statement>
│   │   │   └─ <assignment-statement>
│   │   │       ├── <variable>
│   │   │       │   └─ ident(y)
│   │   │       ├── becomes
│   │   │       └─ <expression>
│   │   │           └─ <simple-expression>
│   │   │               └─ <term>
│   │   │                   └─ <factor>
│   │   │                       └─ intcon(20)
│   │   └─ semicolon
│   └─ <statement>
│       └─ <if-statement>

```

```

<statement>
├── <if-statement>
│   ├── ifsy
│   ├── <expression>
│   │   ├── <simple-expression>
│   │   │   ├── <term>
│   │   │   │   ├── <factor>
│   │   │   │   │   ├── lparent
│   │   │   │   │   ├── <expression>
│   │   │   │   │   │   ├── <simple-expression>
│   │   │   │   │   │   │   ├── <term>
│   │   │   │   │   │   │   │   ├── <factor>
│   │   │   │   │   │   │   │   │   ├── ident(x)
│   │   │   │   │   │   │   │   └── <relational-operator>
│   │   │   │   │   │   │   │   ├── neq
│   │   │   │   │   │   │   │   ├── <simple-expression>
│   │   │   │   │   │   │   │   │   ├── <term>
│   │   │   │   │   │   │   │   │   ├── <factor>
│   │   │   │   │   │   │   │   │   └── ident(y)
│   │   │   │   │   │   │   └── rparent
│   │   │   │   │   ├── <multiplicative-operator>
│   │   │   │   │   │   ├── andsy
│   │   │   │   │   │   ├── <factor>
│   │   │   │   │   │   │   ├── lparent
│   │   │   │   │   │   │   ├── <expression>
│   │   │   │   │   │   │   │   ├── <simple-expression>
│   │   │   │   │   │   │   │   │   ├── <term>
│   │   │   │   │   │   │   │   │   ├── <factor>
│   │   │   │   │   │   │   │   │   │   ├── ident(x)
│   │   │   │   │   │   │   │   │   └── <relational-operator>
│   │   │   │   │   │   │   │   ├── leq
│   │   │   │   │   │   │   │   ├── <simple-expression>
│   │   │   │   │   │   │   │   │   ├── <term>
│   │   │   │   │   │   │   │   │   ├── <factor>
│   │   │   │   │   │   │   │   │   └── ident(y)
│   │   │   │   │   │   └── rparent
│   │   └── thensy
│   └── <statement>
│       ├── <compound-statement>
│       │   ├── beginsy
│       │   ├── <statement-list>
│       │   │   ├── <statement>
│       │   │   │   ├── <assignment-statement>
│       │   │   │   │   ├── <variable>
│       │   │   │   │   │   ├── ident(check)
│       │   │   │   │   ├── becomes
│       │   │   │   │   ├── <expression>
│       │   │   │   │   │   ├── <simple-expression>
│       │   │   │   │   │   │   ├── <term>
│       │   │   │   │   │   │   │   ├── <factor>

```

```

      <term>
      <factor>
      ident(true)
    semicolon
  endsy
elsesy
<statement>
  <compound-statement>
    beginsy
    <statement-list>
      <statement>
        <assignment-statement>
          <variable>
            ident(check)
          becomes
          <expression>
            <simple-expression>
              <term>
                <factor>
                  ident(false)
            semicolon
          endsy
        semicolon
      endsy
    semicolon
  endsy
period

```

```

program CommentTest;
{ Ini adalah komentar satu baris }
(* Ini adalah komentar
   multi-baris Pascal klasik *)
begin
  writeln('Testing quotes: it's working');
  { Komentar di akhir program }
end.

```

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(CommentTest)
│   └── semicolon
├── <declaration-part>
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   ├── <procedure/function-call>
│   │   │   │   ├── ident(writeln)
│   │   │   │   ├── lparent
│   │   │   │   ├── <parameter-list>
│   │   │   │   │   ├── <expression>
│   │   │   │   │   │   ├── <simple-expression>
│   │   │   │   │   │   │   ├── <term>
│   │   │   │   │   │   │   │   ├── <factor>
│   │   │   │   │   │   │   │   │   ├── string('Testing quotes: it's working')
│   │   │   │   │   │   │   └── rparent
│   │   │   └── semicolon
│   │   └── endsy
│   └── period

```

Case 4

```

program ArrayTest;
const
  PI = 3.14;
var
  numbers: array[1..5] of real;
  i: integer;
begin
  for i := 5 downto 1 do
  begin
    numbers[i] := i * PI;
  end;
end.
program RecordTest;
type
  Point = record
    x, y: integer;
  end;
var
  p1: Point;
begin
  p1.x := 10;
  p1.y := 20;

  { Menguji apakah lexer bingung antara . (period) dan .. (range) }
  writeln('Coordinates: ', p1.x, ', ', p1.y);
end.

```

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(ArrayTest)
│   └── semicolon
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── constsy
│   │   ├── ident(PI)
│   │   ├── eql
│   │   ├── <constant>
│   │   │   └── realcon(3.14)
│   │   └── semicolon
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   └── ident(numbers)
│   │   ├── colon
│   │   ├── <type>
│   │   │   ├── <array-type>
│   │   │   │   ├── arraysy
│   │   │   │   ├── lbrack
│   │   │   │   ├── <range>
│   │   │   │   │   ├── <constant>
│   │   │   │   │   │   └── intcon(1)
│   │   │   │   │   ├── period
│   │   │   │   │   ├── period
│   │   │   │   │   └── <constant>
│   │   │   │   │       └── intcon(5)
│   │   │   │   └── rbrack
│   │   │   ├── ofsy
│   │   │   ├── <type>
│   │   │   │   └── ident(real)
│   │   └── semicolon
│   ├── <identifier-list>
│   │   └── ident(i)
│   ├── colon
│   ├── <type>
│   │   └── ident(integer)
│   └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   ├── <for-statement>
│   │   │   │   ├── forsy
│   │   │   │   ├── ident(i)
│   │   │   │   ├── becomes
│   │   │   │   └── <expression>

```

```

├─ forsy
├─ ident(i)
├─ becomes
├─ <expression>
│   └─ <simple-expression>
│       └─ <term>
│           └─ <factor>
│               └─ intcon(5)
├─ downtosy
├─ <expression>
│   └─ <simple-expression>
│       └─ <term>
│           └─ <factor>
│               └─ intcon(1)
├─ dosy
├─ <statement>
│   └─ <compound-statement>
│       └─ beginsy
│           └─ <statement-list>
│               └─ <statement>
│                   └─ <assignment-statement>
│                       └─ <variable>
│                           └─ ident(numbers)
│                               └─ <component-variable>
│                                   └─ lbrack
│                                       └─ <index-list>
│                                           └─ ident(i)
│                                               └─ rbrack
│                               └─ becomes
│                                   └─ <expression>
│                                       └─ <simple-expression>
│                                           └─ <term>
│                                               └─ <factor>
│                                                   └─ ident(i)
│                                               └─ <multiplicative-operator>
│                                                   └─ times
│                                               └─ <factor>
│                                                   └─ ident(PI)
│                               └─ semicolon
│                                   └─ endsy
├─ semicolon
├─ endsy
└─ period

```

Case 5


```

program RecordTest;
type
  Point = record
    x, y: integer;
  end;
var
  p1: Point;
begin
  p1.x := 10;
  p1.y := 20;

  { Menguji apakah lexer bingung antara . (period) dan .. (range) }
  writeln('Coordinates: ', p1.x, ', ', p1.y);
end.

```

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident(RecordTest)
│   └── semicolon
├── <declaration-part>
│   ├── <type-declaration>
│   │   ├── typesy
│   │   ├── ident(Point)
│   │   ├── eql
│   │   ├── <type>
│   │   │   └── <record-type>
│   │   │       ├── recordsy
│   │   │       ├── <field-list>
│   │   │       │   ├── <field-part>
│   │   │       │   │   ├── <identifier-list>
│   │   │       │   │   │   ├── ident(x)
│   │   │       │   │   │   ├── comma
│   │   │       │   │   │   └── ident(y)
│   │   │       │   │   └── colon
│   │   │       │   └── <type>
│   │   │       │       └── ident(integer)
│   │   │       └── semicolon
│   │   └── endsy
│   └── semicolon
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   └── ident(p1)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident(Point)
│   │   └── semicolon
└── <compound-statement>
    ├── beginsy
    ├── <statement-list>
    │   ├── <statement>
    │   │   └── <assignment-statement>
    │   │       ├── <variable>
    │   │       │   ├── ident(p1)
    │   │       │   └── <component-variable>
    │   │       │       ├── period
    │   │       │       └── ident(x)
    │   │       ├── becomes
    │   │       └── <expression>
    │   │           └── <simple-expression>
    │   │               └── <term>
    │   │                   └── <factor>
    │   │                       └── intcon(10)

```

```

<statement>
├─ <assignment-statement>
│   ├── <variable>
│   │   ├── ident(p1)
│   │   └─ <component-variable>
│   │       ├── period
│   │       └─ ident(y)
│   ├── becomes
│   └─ <expression>
│       ├── <simple-expression>
│       └─ <term>
│           └─ <factor>
│               └─ intcon(20)
└─ semicolon

<statement>
├─ <procedure/function-call>
│   ├── ident(writeln)
│   ├── lparen
│   ├── <parameter-list>
│   │   ├── <expression>
│   │   │   ├── <simple-expression>
│   │   │   └─ <term>
│   │   │       └─ <factor>
│   │   │           └─ string('Coordinates: ')
│   │   ├── comma
│   │   ├── <expression>
│   │   │   ├── <simple-expression>
│   │   │   └─ <term>
│   │   │       └─ <factor>
│   │   │           └─ <variable>
│   │   │               ├── ident(p1)
│   │   │               └─ <component-variable>
│   │   │                   ├── period
│   │   │                   └─ ident(x)
│   │   ├── comma
│   │   ├── <expression>
│   │   │   ├── <simple-expression>
│   │   │   └─ <term>
│   │   │       └─ <factor>
│   │   │           └─ charcon(',',')
│   │   ├── comma
│   │   ├── <expression>
│   │   │   ├── <simple-expression>
│   │   │   └─ <term>
│   │   │       └─ <factor>
│   │   │           └─ <variable>
│   │   │               ├── ident(p1)
│   │   │               └─ <component-variable>
│   │   │                   └─ period
└─ .

```

```

└─ <expression>
  └─ <simple-expression>
    └─ <term>
      └─ <factor>
        └─ intcon(20)
└─ semicolon
└─ <statement>
  └─ <procedure/function-call>
    └─ ident(writeln)
    └─ lparent
    └─ <parameter-list>
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ string('Coordinates: ')
      └─ comma
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ <variable>
                └─ ident(p1)
                └─ <component-variable>
                  └─ period
                  └─ ident(x)
      └─ comma
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ charcon(',',')
      └─ comma
      └─ <expression>
        └─ <simple-expression>
          └─ <term>
            └─ <factor>
              └─ <variable>
                └─ ident(p1)
                └─ <component-variable>
                  └─ period
                  └─ ident(y)
    └─ rparent
  └─ semicolon
└─ endsy
└─ period

```

BAB 4 : Kesimpulan dan Saran

4.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi pada fase *Syntax Analysis*, dapat ditarik kesimpulan sebagai berikut:

1. Algoritma *Top-Down Recursive Descent Parser* berhasil diimplementasikan untuk melakukan validasi tata bahasa (*grammar*) secara deterministik. Parser mampu mengonversi aliran token sekuensial menjadi representasi hierarkis memori berbentuk *Parse Tree* 2D.
2. Arsitektur pemetaan satu *node* non-terminal ke dalam satu fungsi mandiri (total 43 fungsi produksi) berhasil menjaga *Cyclomatic Complexity* tetap rendah, mendistribusikan beban validasi sintaks secara terstruktur melalui *Call Stack* C++.
3. Penerapan utilitas *parsing* berbasis expect dan accept memastikan validasi token berjalan ketat sesuai *Context-Free Grammar*. Ketidakcocokan token langsung memicu *Syntax Error* secara presisi.
4. Manajemen memori dinamis pada konstruksi pohon sintaksis telah diamankan menggunakan *Smart Pointers* (`std::unique_ptr`), menghilangkan risiko *memory leak* selama proses pengikatan anak-anak *node* (*children nodes*).
5. Ambiguitas tata bahasa, seperti bentrokan antara penugasan variabel dan pemanggilan prosedur, berhasil direkonstruksi dan diselesaikan menggunakan teknik *Lookahead* ($k=1$) tanpa menggeser *pointer* utama.

4.2 Saran

Untuk pengembangan lebih lanjut dari modul *Syntax Analysis* proyek ini, disarankan beberapa pembaruan teknis berikut:

1. **Implementasi Error Recovery (Panic Mode):** Saat ini, *parser* bersifat *Fail-Fast* yang langsung melakukan terminasi (Halt) saat menemui kesalahan sintaks pertama. Pengembangan selanjutnya perlu menambahkan mekanisme sinkronisasi token (misalnya melompati token hingga menemukan token pembatas seperti semicolon atau endsy) agar kompilator dapat melaporkan banyak kesalahan sintaksis dalam satu kali kompilasi.
2. **Penyederhanaan Menjadi Abstract Syntax Tree (AST):** *Parse Tree* (Concrete Syntax Tree) yang dihasilkan saat ini masih memuat *node* terminal struktural murni seperti tanda kurung (`lparent`, `rparent`) dan titik koma (semicolon). Pohon ini perlu dioptimasi dengan memangkas terminal yang tidak memiliki representasi semantik agar proses pada tahap *Semantic Analysis* menjadi lebih ringan dan cepat.
3. **Peningkatan Kapasitas Lookahead:** Evaluasi prediktif saat ini terbatas pada $k=1$. Jika bahasa pemrograman menerima pembaruan aturan produksi tata bahasa yang lebih

kompleks (*grammar* dengan *left-factoring* panjang), parser perlu ditingkatkan untuk mendukung LL(k) atau *backtracking* agar tidak terjadi kegagalan rekursi.

Lampiran

- Link Repository Github = <https://github.com/kalkabena/XYZ-Tubes-IF2224-2026.git>
- Tabel Pembagian Tugas:

NIM	Nama	Pembagian Tugas	Persentase Kontribusi
13524114	Mirza Tsabita Wafa'ana	Pengaplikasian node for-statement hingga node multiplicative-operator dan mengerjakan laporan	25%
13524119	Nathanael Shane Bennet	Pengaplikasian node statement-list hingga node repeat-statement dan mengerjakan laporan	25%
13524130	Faris Wirakusuma Triawan	Pengaplikasian node program hingga node range, inisialisasi syntax analyzer dan mengerjakan laporan	25%
13524144	Jonathan Harijadi	Pengaplikasian node enumerated hingga node compound-statement dan mengerjakan laporan	25%

Referensi

“An Introduction to Makefiles”

https://www.gnu.org/software/make/manual/html_node/Introduction.html

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Pearson Education. Halaman 109-150 mengenai *Lexical Analysis*.